

Create a service connection to access Azure resources

You will need to create a service connection in Azure DevOps which will allow you to deploy and access resources in your Azure subscription.

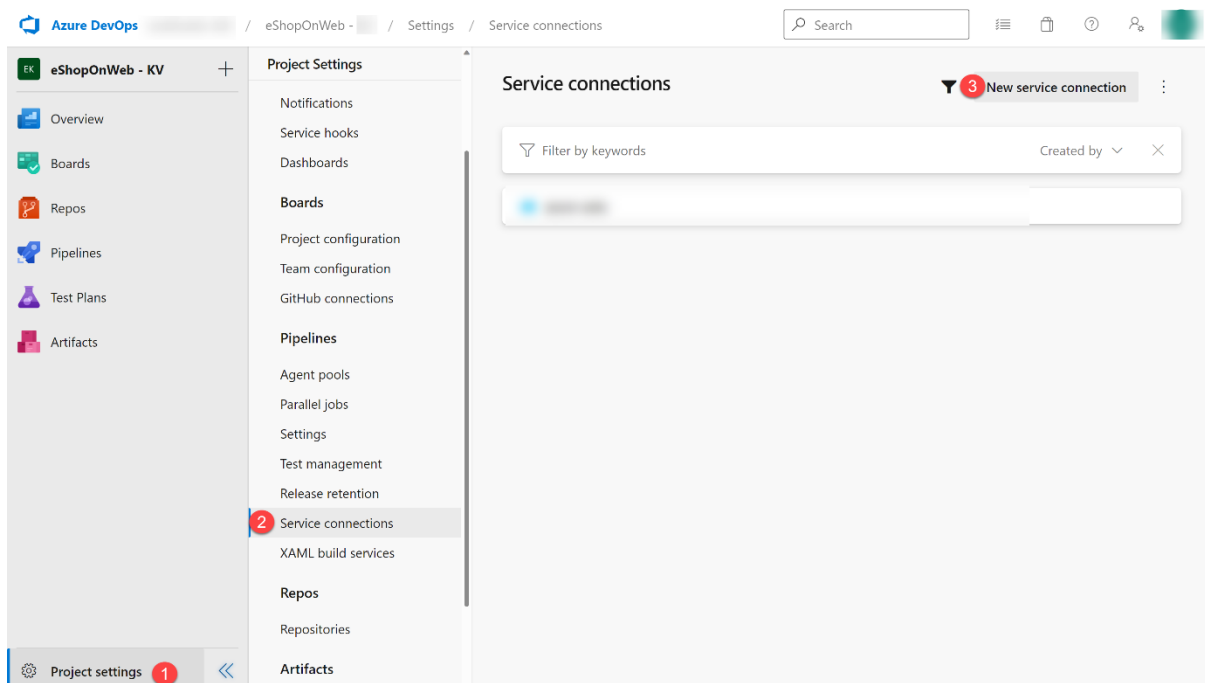
1. Start a web browser, navigate to the Azure DevOps portal <https://aex.dev.azure.com>.
2. Sign in to the Azure DevOps organization with the **Username** and **TAP** present on the **Resources** tab.

In order to determine an **Azure** region with capacity, select a region from the dropdown below and then select the **Check Selected Region** button.

eastuseastus2westuswestus2westus3westeuropecanadacentralsoutheastasiaaustrali
aeast

Once a region is found to have capacity, click the **CreateRG** button to create a Resource Group in this region.

3. Open **eShopOnWeb** project, and select **Project settings** in the bottom left corner of the portal.
4. Select the **Service connections** under Pipelines, and then select **Create service connection** button.



5. On the **New service connection** blade, select **Azure Resource Manager** and **Next** (you may need to scroll down).
6. Select **App registration (automatic)** from the **Identity type** dropbox.

7. Select **Workload Identity federation** and **Subscription** under the **Scope level**.

Note: You can also use **App registration or managed identity (manual)** if you prefer to manually configure the service connection. Follow the steps in the [Azure DevOps documentation](#) to create the service connection manually.

8. Fill in the empty fields using the information:
 - **Subscription:** Select your Azure subscription.
 - **Resource group:** **AZ400-RG161552160**.
 - **Service connection name:** Type **azure subs**. This name will be referenced in YAML pipelines to access your Azure subscription.
9. Make sure the **Grant access permission to all pipelines** option is unchecked and select **Save**.

Important: The **Grant access permission to all pipelines** option is not recommended for production environments. It is only used in this lab to simplify the configuration of the pipeline.

Note: If you see an error message indicating you don't have the necessary permissions to create a service connection, try again, or configure the service connection manually.

You have now completed the necessary prerequisite steps to continue with the labs.

Add User role to Service Connection

1. Select **azure subs** from the **Service connections**.
2. Under **Azure Resource Manager**, Select **Manage App Registration**.
3. Within the App Registration, copy **Display Name** by selecting the **copy to clipboard** icon.
4. In Azure Search Resources, services, and docs, type Subscriptions and select **Subscriptions**.
5. Select the **Ignite-lod52547396** subscription
6. In the left navigation, select **Access Control (IAM)**
7. Select **Add** dropdown and **Add role assignment**
8. Within the **Privileged Administrator Roles**, select LOD Owner, then **next**
9. Select the **Select members** and paste the display name copied from the App Registration into **Search by name or email address**.

10. Select your member ID and save with the **Select** button at the bottom.
11. **Next** to Conditions, select **Allow user to assign all roles (highly privileged)**.
12. Select the **next** button twice and review the assignment.
13. Select **Review + Assign**.

Deploy Docker containers to Azure App Service web apps

In this lab, you'll learn how to use an Azure DevOps CI/CD pipeline to build a custom Docker image, push it to Azure Container Registry, and deploy it as a container to Azure App Service. You'll work with containerization, Azure Container Registry, and automated deployments.

You will learn how to:

- Build a custom Docker image using a Microsoft hosted Linux agent.
- Push an image to Azure Container Registry.
- Deploy a Docker image as a container to Azure App Service using Azure DevOps.
- Configure CI/CD pipelines for containerized applications.
- Test containerized web applications in Azure.

This lab takes approximately **20** minutes to complete.

Before you start

To complete the lab, you need:

- **Microsoft Edge** or an [Azure DevOps supported browser](#).
- An Azure DevOps organization. If you don't already have one, create one by following the instructions at [Create an organization or project collection](#).
- **An Azure subscription:** If you don't already have an Azure subscription, sign up for a free account at [Azure Free Account](#).
- Verify that you have a Microsoft account or a Microsoft Entra account with the **Contributor** or the **Owner** role in the Azure subscription. For details, refer to [List Azure role assignments using the Azure portal](#) and [View and assign administrator roles in Azure Active Directory](#).

Set up Azure DevOps organization (if needed)

In this **Cloudslice** lab, please skip this task.

If you don't already have an Azure DevOps organization, follow these steps:

1. Use a private browser session to get a new **personal Microsoft Account (MSA)** at <https://account.microsoft.com> (skip if you already have one).
2. Using the same browser session, sign up for a free Azure subscription at <https://azure.microsoft.com/free> (skip if you already have one).
3. Open a browser and navigate to Azure portal at <https://portal.azure.com>, then search at the top of the Azure portal screen for **Azure DevOps**. In the resulting page, select **Azure DevOps organizations**.
4. Next, select on the link labelled **My Azure DevOps Organizations** or navigate directly to <https://aex.dev.azure.com>.
5. On the **We need a few more details** page, select **Continue**.
6. In the drop-down box on the left, choose **Default Directory**, instead of **Microsoft Account**.
7. If prompted ("*We need a few more details*"), provide your name, e-mail address, and location and select **Continue**.
8. Back at <https://aex.dev.azure.com> with **Default Directory** selected select the blue button **Create new organization**.
9. Accept the *Terms of Service* by selecting **Continue**.
10. If prompted ("*Almost done*"), leave the name for the Azure DevOps organization at default (it needs to be a globally unique name) and pick a hosting location close to you from the list.
11. Once the newly created organization opens in **Azure DevOps**, select **Organization settings** in the bottom left corner.
12. At the **Organization settings** screen select **Billing** (opening this screen takes a few seconds).
13. Select **Setup billing** and on the right-hand side of the screen, select your **Azure Subscription** and then select **Save** to link the subscription with the organization.

14. Once the screen shows the linked Azure Subscription ID at the top, change the number of **Paid parallel jobs** for **MS Hosted CI/CD** from 0 to **1**. Then select **SAVE** button at the bottom.

Note: You may **wait a couple of minutes before using the CI/CD capabilities** so that the new settings are reflected in the backend. Otherwise, you will still see the message *"No hosted parallelism has been purchased or granted"*.

15. In **Organization Settings**, go to section **Pipelines** and select **Settings**.
16. Toggle the switch to **Off** for **Disable creation of classic build pipelines** and **Disable creation of classic release pipelines**.
17. In **Organization Settings**, go to section **Security** and select **Policies**.
18. Toggle the switch to **On** for **Allow public projects**.

Create and configure the Azure DevOps project (if needed)

In this **Cloudslice** lab, please skip this task.

1. Open your browser and navigate to your Azure DevOps organization.
2. Select the **New Project** option and use the following settings:
 - name: **eShopOnWeb**
 - visibility: **Private**
 - Advanced: Version Control: **Git**
 - Advanced: Work Item Process: **Scrum**
3. Select **Create**.

Import eShopOnWeb git repository (if needed)

In this **Cloudslice** lab, please skip this task.

1. Open the previously created **eShopOnWeb** project.
2. Select the **Repos > Files, Import a Repository** and then select **Import**.
3. On the **Import a Git Repository** window, paste the following URL <https://github.com/MicrosoftLearning/eShopOnWeb.git> and select **Import**:
4. The repository is organized the following way:
 - **.ado** folder contains Azure DevOps YAML pipelines.
 - **.devcontainer** folder container setup to develop using containers (either locally in VS Code or GitHub Codespaces).

- **infra** folder contains Bicep & ARM infrastructure as code templates used in some lab scenarios.
 - **.github** folder contains YAML GitHub workflow definitions.
 - **src** folder contains the .NET 8 website used in the lab scenarios.
5. Leave the web browser window open.
 6. Go to **Repos > Branches**.
 7. Hover on the **main** branch then select the ellipsis on the right of the column.
 8. Select on **Set as default branch**.

Create a service connection to access Azure resources (if needed)

You will need to create a service connection in Azure DevOps which will allow you to deploy and access resources in your Azure subscription.

1. Start a web browser, navigate to the Azure DevOps portal <https://aex.dev.azure.com>.
2. Sign in to the Azure DevOps organization.

In order to determine an **Azure** region with capacity, select a region from the dropdown below and then select the **Check Selected Region** button.

eastuseastus2westuswestus2westus3westeuropcanadacentralsoutheastasiaaustrali
aeast

Once a region is found to have capacity, click the **CreateRG** button to create a Resource Group in this region.

3. Open **eShopOnWeb** project, and select **Project settings** in the bottom left corner of the portal.
4. Select the **Service connections** under Pipelines, and then select **Create service connection** button.
5. On the **New service connection** blade, select **Azure Resource Manager** and **Next** (you may need to scroll down).
6. Select **App registration (automatic)** from the **Identity type** dropbox.
7. Select **Workload Identity federation** and **Subscription** under the **Scope level**.

Note: You can also use **App registration or managed identity (manual)** if you prefer to manually configure the service connection. Follow the steps in the [Azure DevOps documentation](#) to create the service connection manually.

8. Fill in the empty fields using the information:
 - **Subscription:** Select your Azure subscription.
 - **Resource group:** **AZ400-RG161552160**.
 - **Service connection name:** Type **azure subs**. This name will be referenced in YAML pipelines to access your Azure subscription.
9. Make sure the **Grant access permission to all pipelines** option is unchecked and select **Save**.

Important: The **Grant access permission to all pipelines** option is not recommended for production environments: selecting the option means granting access to the service connection to all pipelines in the project, not selecting the option allows you to approve the access to the service connection on the first run of each pipeline.

****Note**:** If the ****Grant access permission to all pipelines**** option is disabled (it is in gray) and cannot be changed, continue with the lab.

****Note**:** If you see an error message indicating you don't have the necessary permissions to create a service connection, try again, or configure the service connection manually.

Import and run the CI pipeline

In this section, you will configure the service connection with your Azure Subscription then import and run the CI pipeline.

Import and run the CI pipeline

1. Go to **Pipelines > Pipelines**
2. Select on **New pipeline** button (or **Create Pipeline** if you don't have other pipelines previously created)
3. Select **Azure Repos Git (YAML)**
4. Select the **eShopOnWeb** repository
5. Select **Existing Azure Pipelines YAML file**
6. Select the **main** branch and the **/.ado/eshoponweb-ci-docker.yml** file, then select on **Continue**

7. In the YAML pipeline definition, customize:
 - **YOUR-SUBSCRIPTION-ID** with your Azure subscription ID edd5b733-e0c3-4f2d-9c53-1c9f46c3583e
 - Replace the **resourceGroup** with **AZ400-RG161552160**
 - Replace **location** with westeurope
8. Review the pipeline definition. The CI definition consists of the following tasks:
 - **Resources:** It downloads the repository files that will be used in the following tasks.
 - **AzureResourceManagerTemplateDeployment:** Deploys the Azure Container Registry using bicep template.
 - **PowerShell:** Retrieve the **ACR Login Server** value from the previous task's output and create a new parameter **acrLoginServer**
 - **Docker - Build:** Build the Docker image and create two tags (Latest and current BuildID)
 - **Docker - Push:** Push the images to Azure Container Registry
9. Select on **Save and Run**.
10. Open the pipeline execution. If you see a warning message "This pipeline needs permission to access a resource before this run can continue to Build", select on **View** and then **Permit** and **Permit** again. This will allow the pipeline to access the Azure subscription.

Note: The deployment may take a few minutes to complete.

11. Your pipeline will take a name based on the project name. Let's **rename** it for identifying the pipeline better. Go to **Pipelines > Pipelines** and select on the recently created pipeline. Select on the ellipsis and **Rename/move** option. Name it **eshoponweb-ci-docker** and select on **Save**.
12. Navigate to the **Azure Portal** <https://portal.azure.com>, search for the Azure Container Registry in the recently created Resource Group (it should be named **AZ400-RG161552160**). On the left-hand side select **Repositories** under **Services** and make sure that the repository **eshoponweb/web** was created. When you select the repository link, you should see two tags (one of them is **latest**), these are the pushed images. If you don't see this, check the status of your pipeline.

Import and run the CD pipeline

In this section, you will configure the service connection with your Azure Subscription then import and run the CD pipeline.

Import and run the CD pipeline

In this task, you will import and run the CD pipeline.

1. Go to **Pipelines > Pipelines**
2. Select on **New pipeline** button
3. Select **Azure Repos Git (YAML)**
4. Select the **eShopOnWeb** repository
5. Select **Existing Azure Pipelines YAML File**
6. Select the **main** branch and the **/.ado/eshoponweb-cd-webapp-docker.yml** file, then select on **Continue**
7. In the YAML pipeline definition, customize:
 - **YOUR-SUBSCRIPTION-ID** with your Azure subscription ID edd5b733-e0c3-4f2d-9c53-1c9f46c3583e
 - Replace the **resourceGroup** with **AZ400-RG161552160**
 - Replace **location** with westeurope
8. Review the pipeline definition. The CD definition consists of the following tasks:
 - **Resources:** It downloads the repository files that will be used in the following tasks.
 - **AzureResourceManagerTemplateDeployment:** Deploys the Azure App Service using bicep template.
 - **AzureResourceManagerTemplateDeployment:** Add role assignment using Bicep
9. Select on **Save and Run**.
10. Open the pipeline execution. If you see a warning message "This pipeline needs permission to access a resource before this run can continue to Deploy", select on **View** and then **Permit** and **Permit** again. This will allow the pipeline to access the Azure subscription.

Important: If you do not authorize the pipeline when configuring, you will encounter permission errors during execution. Common error messages include "This pipeline

needs permission to access a resource" or "Pipeline run failed due to insufficient permissions". To resolve this, navigate to the pipeline run, select **View** next to the permission request, then select **Permit** to grant the necessary access to your Azure subscription and resources.

Note: The deployment may take a few minutes to complete.

If you receive the error message "TF402455: Pushes to this branch are not permitted; you must use a pull request to update this branch.", you need to uncheck the "Require a minimum number of reviewers" Branch protection rule enabled in the previous labs.

11. Your pipeline will take a name based on the project name. Let's **rename** it for identifying the pipeline better. Go to **Pipelines > Pipelines** and hover on the recently created pipeline. Select on the ellipsis and **Rename/move** option. Name it **eshoponweb-cd-webapp-docker** and select on **Save**.

Note 1: The use of the **/infra/webapp-docker.bicep** template creates an app service plan, a web app with system assigned managed identity enabled, and references the Docker image pushed previously: **\${acr.properties.loginServer}/eshoponweb/web:latest**.

Note 2: The use of the **/infra/webapp-to-acr-roleassignment.bicep** template creates a new role assignment for the web app with AcrPull role to be able to retrieve the Docker image. This could be done in the first template, but since the role assignment can take some time to propagate, it's a good idea to do both tasks separately.

Test the solution

1. In the Azure Portal, navigate to the recently created Resource Group, you should now see three resources (App Service, App Service Plan and Container Registry).
2. Navigate to the App Service, then select **Browse**, this will take you to the website.
3. Verify that the eShopOnWeb application is running successfully. Once confirmed, you have completed the lab successfully.

Clean up resources

When you complete the lab, it's important to clean up your Azure resources to avoid unnecessary charges:

Delete the Azure resources

1. In the Azure Portal at <https://portal.azure.com>, navigate to the **Resource groups** section.
2. Find and select the **AZ400-RG1** resource group (or the name you used).
3. On the resource group page, select **Delete resource group**.
4. Type the resource group name to confirm deletion and select **Delete**.
5. Wait for the deletion process to complete.

Clean up Azure DevOps resources

You don't need to clean up your Azure DevOps organization or project, as they will remain available for you to use as a reference and portfolio item. Azure DevOps provides free tier usage that includes basic features for small teams.

If you want to delete the project, you can do so by following these steps:

1. In your browser navigate to the Azure DevOps portal at <https://aex.dev.azure.com>.
2. Navigate to the **eShopOnWeb** project you created.
3. On the project settings page, go to **Overview** and select **Delete** at the bottom of the page.
4. Type the project name to confirm deletion and select **Delete**.

CAUTION: Deleting a project deletes all work items, repositories, builds, and other project artifacts. If you used an existing project for this exercise, any existing resources outside the scope of this exercise will also be deleted.

IMPORTANT: Remember to delete the Azure resources to avoid unnecessary charges. The Azure DevOps project can remain as part of your portfolio.